Nama: ALDI SAPUTRA

NIM: 105841115923

Pertemuan 02: Data Preprocessing dan EDA

Tujuan Praktikum: Di praktikum kedua ini, fokus utamanya adalah belajar gimana caranya "membersihkan" dan merapikan data mentah (Data Preprocessing) sebelum dimasukin ke model Machine Learning. Selain itu, ada juga sesi EDA (Exploratory Data Analysis) buat membedah isi datanya lewat grafik biar kita tahu wawasan (insight) apa saja yang tersembunyi di dalamnya.Penjelasan Kode Ringkas (Pemanasan)

Penjelasan Langkah Kerja

```
import matplotlib
matplotlib.use("Agg") # Mode non-interaktif untuk penyimpanan file

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

# ============================================================
# Memuat dataset Titanic
# ============================================================
df = sns.load_dataset("titanic")
print("=== Ukuran Data Awal ===")
print(f"Shape: {df.shape}")
print("\nTipe kolom:")
print(df.dtypes)
```

```
=== Ukuran Data Awal ===
Shape: (891, 15)

Tipe kolom:
survived          int64
pclass            int64
sex              object
age             float64
sibsp             int64
parch             int64
fare            float64
embarked         object
class          category
who              object
adult_male         bool
deck           category
embark_town      object
alive            object
alone              bool
dtype: object
```

Langkah pertama saya buka dengan me-*load* library seperti Pandas, NumPy, Matplotlib, Seaborn, dan Scikit-learn (khususnya StandardScaler). Saya juga tetap pakai setelan matplotlib.use("Agg") supaya aman pas nyimpen grafik. Dataset yang dipakai kali ini adalah

dataset penumpang kapal "Titanic" yang langsung ditarik dari library Seaborn pakai perintah sns.load_dataset("titanic"). Pas dicek bentukannya pakai .shape, ternyata ukuran data awalnya lumayan besar, yaitu ada 891 baris dan 15 kolom.

1. Penanganan Data Kosong (*Missing Value*)

```
# 1. Missing Value
print("\n=== Missing Value (sebelum) ===")
missing_before = df.isnull().sum()
print(missing_before[missing_before > 0])

# Strategi pengisian:
# - Age: Median (karena distribusi mungkin miring)
# - Embarked/Embark Town: Mode (kategori paling sering muncul)
# - Deck: Kategori baru "Unknown" (karena terlalu banyak yang hilang)
df["age"] = df["age"].fillna(df["age"].median())
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])
df["embark_town"] = df["embark_town"].fillna(df["embark_town"].mode()[0])

# Penanganan khusus untuk kolom kategori 'deck'
if 'deck' in df.columns:
    df["deck"] = df["deck"].cat.add_categories("Unknown").fillna("Unknown")

print("\n=== Missing Value (sesudah) ===")
missing_after = df.isnull().sum()
print(missing_after[missing_after > 0] if missing_after.sum() > 0 else "Tidak ada missing value tersisa.")
```

```
=== Missing Value (sebelum) ===
age            177
embarked         2
deck           688
embark_town      2
dtype: int64

=== Missing Value (sesudah) ===
Tidak ada missing value tersisa.
```

Pas saya cek kelengkapan datanya pakai df.isnull().sum(), ternyata ada banyak data yang bolong alias kosong. Kolom age (umur) kehilangan 177 data, kolom embarked dan embark_town hilang 2 data, dan yang paling parah kolom deck kosong sampai 688 baris.

Buat ngakalin ini, saya pakai strategi beda-beda:

- Untuk kolom age, karena datanya berupa angka, celahnya saya tambal pakai nilai *Median* (nilai tengah) biar nggak gampang terpengaruh angka ekstrem.
- Untuk embarked dan embark_town (kategori), saya tambal pakai *Mode* atau data yang paling sering muncul.
- Nah, buat deck karena yang bolong terlalu banyak, saya bikinin aja satu kategori baru bernama "Unknown" buat ngisi kolom yang kosong itu. Setelah baris kode ini di-*run*, semua *missing value* langsung bersih tak tersisa.

2. Menghapus Data Duplikat

```python
# 2. Duplikat
print("\n=== Duplikat ===")
duplicates_count = df.duplicated().sum()
print(f"Jumlah duplikat ditemukan: {duplicates_count}")
df = df.drop_duplicates()
print(f"Shape setelah hapus duplikat: {df.shape}")
```

```
...
=== Duplikat ===
Jumlah duplikat ditemukan: 110
Shape setelah hapus duplikat: (781, 15)
```

Langkah selanjutnya saya nyari data yang dobel atau kembar pakai fungsi .duplicated().sum(). Hasilnya, sistem nemuin ada 110 baris data yang duplikat. Karena data dobel ini bisa bikin model ML nanti jadi bias, akhirnya langsung saya buang semuanya pakai df.drop_duplicates(). Alhasil, ukuran baris data saya nyusut dari 891 sisa 781 baris.

3. Mengatasi Outlier (Pencilan Data)

```python
# 3. Outlier (IQR) pada kolom fare dan age
print("\n=== Outlier (IQR) ===")
for col in ["fare", "age"]:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    low = q1 - 1.5 * iqr
    high = q3 + 1.5 * iqr

    outliers_before = ((df[col] < low) | (df[col] > high)).sum()
    # Clipping/Capping: Membatasi nilai ke batas low/high
    df[col] = df[col].clip(low, high)
    print(f"  {col.capitalize()}: {outliers_before} outlier diproses (batas {low:.1f} - {high:.1f})")
```

```
...
=== Outlier (IQR) ===
  Fare: 102 outlier diproses (batas -30.9 - 73.0)
  Age: 39 outlier diproses (batas 1.0 - 57.0)
```

Data yang angkanya terlalu nyeleneh (outlier) biasanya bikin kacau hitungan. Di sini saya ngecek outlier di kolom fare (harga tiket) dan age (umur) pakai rumus IQR (Interquartile Range). Setelah dapat batas bawah (*low*) dan batas atas (*high*)-nya, saya nggak ngehapus datanya, melainkan saya pangkas alias dibatasi (*clipping*) pakai fungsi .clip(low, high). Hasil eksekusinya, ada 102 nilai outlier di harga tiket dan 39 nilai outlier di umur yang berhasil dijinakkan masuk ke batas normal.

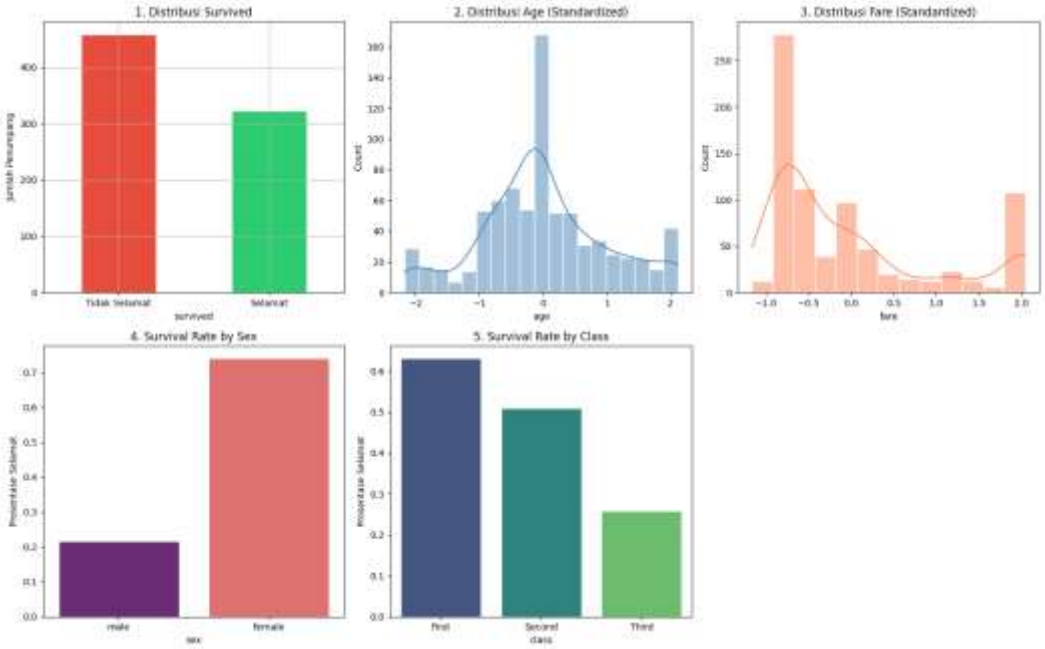
4. Scaling (Standarisasi Data)

```
# 4. Scaling (Standardization)
# Mengubah distribusi menjadi mean=0 dan std=1
scaler = StandardScaler()
df[["age", "fare"]] = scaler.fit_transform(df[["age", "fare"]])

print("\n=== Statistik Setelah Scaling ===")
print(df[["age", "fare"]].describe().round(3))


=== Statistik Setelah Scaling ===
            age     fare
count  781.000  781.000
mean     0.000   -0.000
std      1.001    1.001
min     -2.170   -1.167
25%     -0.562   -0.812
50%     -0.103   -0.466
75%      0.510    0.333
max      2.117    2.050
```

Biar model ML gampang nyerna angka-angkanya, skala data di kolom age dan fare saya seragamkan pakai StandardScaler dari Scikit-learn. Habis di-*scaling*, nilai rata-rata (*mean*) kedua kolom itu berubah jadi 0 (atau mendekati 0), dan standar deviasinya jadi 1.

Exploratory Data Analysis (EDA) & Insight

```
# 6. EDA — 5 Visualisasi + 5 Insight
# ==================================================

# Menyiapkan canvas visualisasi
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
sns.set_style("whitegrid")

# Visualisasi 1: Distribusi Target (Survived)
df["survived"].value_counts().plot(kind="bar", ax=axes[0, 0], color=["#e74c3c", "#2ecc71"])
axes[0, 0].set_title("1. Distribusi Survived")
axes[0, 0].set_xticklabels(["Tidak Selamat", "Selamat"], rotation=0)
axes[0, 0].set_ylabel("Jumlah Penumpang")

# Visualisasi 2: Distribusi Umur (Setelah Scaling)
sns.histplot(df["age"], kde=True, ax=axes[0, 1], color="steelblue")
axes[0, 1].set_title("2. Distribusi Age (Standardized)")

# Visualisasi 3: Distribusi Fare (Setelah Scaling)
sns.histplot(df["fare"], kde=True, ax=axes[0, 2], color="coral")
axes[0, 2].set_title("3. Distribusi Fare (Standardized)")

# Visualisasi 4: Survival Rate berdasarkan Gender
sns.barplot(x="sex", y="survived", data=df, ax=axes[1, 0], palette="magma", ci=None)
axes[1, 0].set_title("4. Survival Rate by Sex")
axes[1, 0].set_ylabel("Prosentase Selamat")

# Visualisasi 5: Survival Rate berdasarkan Class
sns.barplot(x="class", y="survived", data=df, ax=axes[1, 1], palette="viridis", ci=None)
axes[1, 1].set_title("5. Survival Rate by Class")
axes[1, 1].set_ylabel("Prosentase Selamat")

# Menghapus subplot kosong (baris 2, kolom 3)
axes[1, 2].axis("off")

plt.tight_layout()
plt.savefig("pertemuan02_eda_lengkap.png")
plt.close()
print("\nVisualisasi disimpan ke: pertemuan02_eda_lengkap.png")

# 5 Insight Berdasarkan Data
print("\n=== 5 Insight Analisis ===")
total_survived = df["survived"].mean()
female_surv = df[df['sex']=='female']['survived'].mean()
male_surv = df[df['sex']=='male']['survived'].mean()
pclass1_surv = df[df['class']=='First']['survived'].mean()

print(f"1. Tingkat survival rata-rata penumpang adalah {total_survived:.1%}.")
print(f"2. Gender Gap: Survival rate perempuan ({female_surv:.1%}) jauh lebih tinggi "
      f"dibanding laki-laki ({male_surv:.1%}), menunjukkan kebijakan 'ladies first'.")
print(f"3. Hierarki Sosial: Penumpang kelas 1 memiliki peluang selamat tertinggi ({pclass1_surv:.1%}) "
      f"dibandingkan kelas lainnya.")
print(f"4. Preprocessing Skala: Fitur 'age' dan 'fare' kini memiliki rata-rata ≈0, "
      f"yang akan mempercepat konvergensi model Machine Learning nantinya.")
print(f"5. Dampak Outlier: Dengan metode clipping pada 'fare', model tidak akan terganggu "
      f"oleh harga tiket ekstrem yang biasanya merupakan anomali.")

print("\n✅ Pengerjaan Pertemuan 02 Selesai.")
```



Setelah datanya bersih, saya bikin kanvas gede plt.subplots(2, 3) buat nampilin 5 grafik yang berbeda sekaligus dan menyimpannya dengan nama pertemuan02_eda_lengkap.png. Grafiknya macam-macam, ada *bar chart* buat lihat perbandingan yang selamat dan nggak selamat, ada histogram distribusi umur dan harga tiket setelah di-*scaling*, sampai grafik perbandingan *survival rate* berdasarkan jenis kelamin (Sex) dan kelas penumpang (Class).

Berdasarkan *output* perhitungan datanya, saya nulis 5 buah *insight* (kesimpulan analisis) sebagai berikut:

1. Ternyata rata-rata tingkat keselamatan penumpang Titanic di dataset ini cuma sekitar 41.4%.

2. Ada *Gender Gap* yang kerasa banget. Penumpang perempuan peluang selamatnya sampai 74.1%, jauh ninggalin laki-laki yang cuma 21.7%. Ini nunjukin kalau aturan *"ladies first"* pas evakuasi sekoci beneran diterapin.

3. Kelas sosial juga ngaruh. Penumpang tiket Kelas 1 (First class) punya *survival rate* paling tinggi (63.1%) dibanding kelas ekonomi di bawahnya.

4. Proses *Scaling* pada kolom age dan fare berhasil bikin nilai tengahnya jadi 0, ini bakal sangat ngebantu ngeringanin beban komputasi pas nanti kita *training* model ML.

5. Gara-gara nilai ekstrem di harga tiket (*fare*) udah di-*clip* (dipangkas) di proses *Outlier* tadi, model kita ke depannya bakal lebih tahan banting dan nggak gampang bias sama anomali harga tiket yang kemahalan.

Kesimpulan Akhir

Dari praktikum kedua ini, saya jadi sadar kalau data mentah itu ternyata berantakan banget. Tahapan *Preprocessing* kayak ngisi data kosong, buang duplikat, motong outlier, sampai *scaling* itu mutlak dilakuin supaya data yang disuapin ke AI/Machine Learning nanti benar-benar matang dan berkualitas. Lewat EDA, saya juga jadi gampang baca cerita atau fakta unik di balik ribuan baris angka tersebut lewat bantuan grafik dan visualisasi.